

Introducción a Spring Boot

Construyendo un Tracker de Actividad de GitHub

Clase de Introducción a Spring Boot

18 de febrero de 2026

¿Qué es Spring Boot?

- Spring Boot es una extensión del framework Spring que facilita la creación de aplicaciones stand-alone y de producción.
- **Filosofía principal:** “Convention over Configuration” (Convención sobre Configuración).
- Minimiza la configuración manual necesaria para empezar un proyecto.
- Incluye un servidor web embebido (Tomcat, Jetty, Undertow), por lo que no necesitas desplegar archivos WAR.

Spring Boot vs Java EE (Jakarta EE)

Java EE / Jakarta EE

- Requiere servidor de aplicaciones externo (JBoss, WebLogic).
- Configuración pesada (XML, aunque ha mejorado).
- Ciclo de desarrollo más lento.

Spring Boot

- Servidor embebido (Run as JAR).
- Configuración automática.
- Ecosistema enorme y desarrollo rápido.
- Ideal para Microservicios.

Spring Boot utiliza "Starters" para agrupar dependencias comunes.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version>
</parent>

<dependencies>
  <!-- Starter para aplicaciones web (incluye Tomcat, Spring MVC) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Librería para pruebas -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

La Clase Principal

El punto de entrada es una clase Java estándar con la anotación `@SpringBootApplication`.

```
package com.example.githubtracker;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GithubTrackerApplication {

    public static void main(String[] args) {
        SpringApplication.run(GithubTrackerApplication.class, args);
    }
}
```

Caso de Estudio: GitHub Activity Tracker

Vamos a reimaginar nuestra CLI de JavaScript como una API REST en Spring Boot.

Objetivo:

- Recibir peticiones en `/api/activity/username`.
- Consultar la API de GitHub.
- Devolver la actividad formateada o en JSON.

Arquitectura de 3 Capas:

- 1 **Controller:** Maneja las peticiones HTTP.
- 2 **Service:** Contiene la lógica de negocio (Llamada a GitHub).
- 3 **Model:** Representa los datos (Eventos, Usuario).

1. Model (POJOs)

Definimos clases simples para mapear la respuesta JSON de GitHub.

```
public class GitHubEvent {  
    private String id;  
    private String type;  
    private Actor actor;  
    private Repo repo;  
  
    // Getters, Setters, Constructor  
}  
  
public class Actor {  
    private String login;  
    // ...  
}  
  
public class Repo {  
    private String name;  
    // ...  
}
```


2. Service (Lógica de Negocio)

Usamos `@Service` para definir un bean de Spring. Usamos `RestTemplate` para llamadas HTTP.

`@Service`

```
public class GitHubService {  
  
    private final RestTemplate restTemplate;  
    private final String GITHUB_API_URL = "https://api.g  
  
    public GitHubService(RestTemplateBuilder builder) {  
        this.restTemplate = builder.build();  
    }  
  
    public GitHubEvent[] fetchActivity(String username)  
        String url = String.format(GITHUB_API_URL, usern  
        return restTemplate.getForObject(url, GitHubEven  
    }  
}
```

3. Controller (API REST)

Usamos `@RestController` y `@GetMapping` para exponer los endpoints.

```
@RestController
@RequestMapping("/api/activity")
public class ActivityController {

    @Autowired
    private GitHubService gitHubService;

    @GetMapping("/{username}")
    public List<String> getUserActivity(@PathVariable String username) {
        GitHubEvent[] events = gitHubService.fetchActivity(username);
        return formatEvents(events); // Metodo auxiliar
    }

    private List<String> formatEvents(GitHubEvent[] events) {
        // Logica similar a nuestro activity-formatter.js
        return Arrays.stream(events)
            .map(event -> "-" + event.getType() + " en " + event.getUsername())
            .collect(Collectors.toList());
    }
}
```

Configuración (application.properties)

Podemos externalizar configuración como la URL de la API.

src/main/resources/application.properties

```
server.port=8081
github.api.url=https://api.github.com/users/%s/events
logging.level.org.springframework.web=DEBUG
```

Uso en Java:

```
@Value("${github.api.url}")
private String apiUrl;
```

- **Inversión de Control (IoC):** Spring gestiona el ciclo de vida de los objetos (Beans).
- **Inyección de Dependencias (DI):** @Autowired o inyección por constructor conecta los componentes (Controller → Service).
- **Simplicidad:** Hemos migrado la lógica de nuestra CLI a una aplicación web escalable con muy poco código "boilerplate".
- **Siguientes pasos:**
 - Manejo de Errores (@ControllerAdvice).
 - Tests Unitarios (@SpringBootTest).
 - Persistencia de datos (Spring Data JPA).

¡Gracias!

¿Preguntas?